Sup de Vinci
Master 2 Expert en Développement Logiciel

Rapport d'audit de performance

Projet Poule Renard Vipère

RNCP Bloc 4, activité 13
Optimisations et performances backend

Timéo CHERBONNEL

M2 Dev B, session E42
Encadré par Christophe Lecroq
Octobre 2026

1. Présentation du projet

Pour le projet noté j'ai choisi un sujet libre, Poule Renard Vipère. C'est un simulateur écrit en Go qui reprend l'architecture du TP fil rouge ZombieHorde. Trois espèces de 3 000 agents se chassent en cycle. La poule attrape la vipère, le renard attrape la poule et la vipère attrape le renard. Les agents capturés vont en prison et leurs coéquipiers peuvent venir les libérer. Le serveur calcule la partie à 30 ticks par seconde et envoie l'état au navigateur par WebSocket. Il peut aussi jouer des tournois sans interface et les enregistrer dans PostgreSQL.

Le hot path est la fonction `World.Update`, qui fait avancer la partie d'un tick. Pour tenir 30 TPS, un tick doit durer moins de 33,3 ms.

Mon projet avait déjà quelques optimisations faites pendant les TP. Pour pouvoir mesurer chaque étape séparément, j'ai remis les versions naïves dans le code derrière des tags de build (`naive_grid`, `naive_alloc`, `naive_layout` et `naive_trig`). La version V0 active tous ces tags et chaque version suivante en enlève un. Toutes les versions jouent exactement la même partie, avec le même nombre de captures et d'agents libres après 300 ticks, donc on compare bien le même travail.

Toutes les mesures se relancent avec le script `./run_benchmarks.sh` et les résultats bruts sont dans le dossier `bench/` du dépôt.

2. Environnement et protocole de mesure

Composant	Détail
Machine	PC portable HP Victus 15
CPU	Intel Core i5-12450H avec 8 cœurs (4 performants avec hyper-threading et 4 économes), 12 threads, jusqu'à 4,4 GHz
Cache L1	48 Ko de données et 32 Ko d'instructions par cœur performant, 32 Ko et 64 Ko par cœur économe
Cache L2	1,25 Mo par cœur performant et 2 Mo partagés par les 4 cœurs économes
Cache L3	12 Mo partagés
RAM	16 Go DDR4-3200 en double canal (2 barrettes de 8 Go)
Système	Windows 11 Famille 10.0.26200
Runtime	Go 1.27.1 avec GOMAXPROCS à 12 et GOGC à 100
Outils	hyperfine 1.20.0, benchstat, pprof, fieldalignment et PostgreSQL 17 dans Docker

Pour limiter le bruit, le PC était branché sur secteur avec le mode d'alimentation Windows en meilleures performances. Docker était coupé pendant les mesures CPU et je ne lançais qu'une mesure à la fois. Le script enregistre l'état de la batterie et le mode d'alimentation dans `bench/env.txt` à chaque lancement. J'ai ajouté ça parce que mes premières mesures avaient été faites sur batterie sans que je m'en rende compte. Le même code donnait alors 6,30 ms par tick au lieu de 2,31 ms sur secteur, donc j'ai tout refait.

Le benchmark Go `BenchmarkUpdate` mesure trois moments de la partie, le début, le milieu (tick 1 500 avec toutes les règles) et le milieu sans les règles optionnelles que j'appelle classique. L'état de départ est restauré avant chaque passe, donc chaque passe rejoue les mêmes 30 ticks. Je fais 10 passes par version avec `-count=10` et je les compare avec benchstat, qui donne la médiane et un intervalle de confiance à 95 %. Je ne garde une

différence que si p est inférieur à 0,05. J'ai aussi ajouté le nombre de cycles de GC par tick, lu avec `runtime/metrics`.

Pour le binaire complet j'utilise hyperfine avec la commande `hyperfine -N --warmup 3 --runs 10 "vN.exe -ticks 300 -workers 1"`, qui joue les 300 premiers ticks d'une partie. hyperfine donne la moyenne, la médiane et l'écart-type, et un petit outil du dépôt (`tools/hfstats`) calcule en plus la variance à partir des temps bruts.

3. Diagnostic de la version de départ

V0 avec 1 worker	Début	Milieu	Classique
Temps par tick	135,8 ms	117,5 ms	70,8 ms
Mémoire allouée par tick	105,8 Mio	102,0 Mio	59,0 Mio
Cycles de GC par tick	47	33	15

La V0 ne tient pas du tout le temps réel. Un tick prend 135,8 ms au début de la partie, ce qui fait environ 7 TPS au lieu de 30.

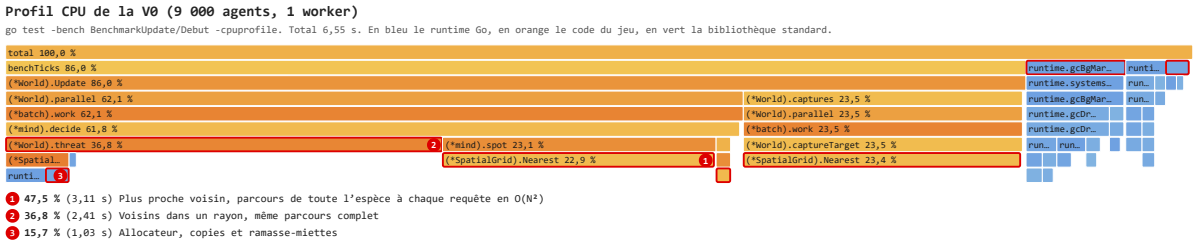


Figure 1. Profil CPU de la V0 obtenu avec pprof et annoté

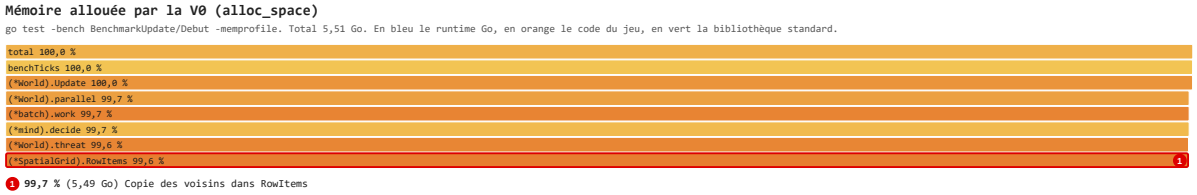


Figure 2. Profil des allocations de la V0, presque tout vient de RowItems

Dans le profil CPU, 47,5 % du temps est passé dans `SpatialGrid.Nearest`, qui cherche la proie la plus proche, et 36,8 % dans `threat` et les autres recherches de voisins. Dans la version naïve ces fonctions parcourent toute l'espèce à chaque requête. J'ai compté les appels dans une copie instrumentée du code et j'arrive à 81 millions de calculs de distance par tick. Il y a 18 000 appels à `Nearest` qui examinent 3 000 candidats chacun, plus 9 000 appels à `RowItems` qui en renvoient 3 000 aussi. Chaque calcul ne fait que deux soustractions, deux multiplications et une comparaison mais il est répété 81 millions de fois.

Le profil mémoire montre le deuxième problème. `RowItems` renvoie une copie de la liste des voisins à chaque appel, soit 12 Ko par appel et plus de 100 Mo alloués par tick. Comme le tas vivant ne fait que quelques Mo, le GC se déclenche en permanence, avec 47 cycles par tick.

Le hot path initial est donc la recherche de voisins en O(N²) dans `Nearest` et `RowItems`, qui représente 84 % du CPU. Vient ensuite la pression sur le GC causée par les copies de `RowItems`.

4. Journal d'optimisation

Version	Changement	Début	Milieu	Classique	GC par tick	Écart au milieu
V0	version naïve	135,8 ms	117,5 ms	70,8 ms	33	
V1	grille spatiale	2,405 ms	2,374 ms	1,962 ms	0,3	-98,0 % (p = 0,000)
V2	zéro allocation	1,923 ms	1,800 ms	1,585 ms	0	-24,2 % (p = 0,000)
V3	struct de 64 octets	1,923 ms	1,799 ms	1,585 ms	0	non significatif (p = 0,80)
V4	math.Sincos	1,919 ms	1,794 ms	1,694 ms	0	non significatif (p = 0,74)
V5	pool de 4 workers	0,880 ms	1,086 ms	0,901 ms	0	-39,5 % (p = 0,000)

Tableau 1. Médianes de 10 passes comparées avec benchstat, V0 à V4 sur 1 worker et V5 sur 4

4.1 Mémoire et localité de cache

Pour la V1 j'ai mis en place une grille spatiale. L'arène est découpée en 100 par 100 cellules de 50 unités et à chaque tick les agents sont rangés par cellule avec un tri par comptage en O(N). Tout est stocké dans un seul tableau `[]int32` où les cellules d'une même rangée se suivent. Une recherche ne lit plus que quelques dizaines d'agents au lieu de 3 000 et elle lit des zones contiguës en mémoire, ce qui aide le préchargement du processeur. Un test vérifie sur 67 500 requêtes que la grille renvoie exactement le même agent que le parcours complet. Le tick baisse de 98 % et c'est de loin l'optimisation qui a le plus compté.

Le profil de la V1 montrait encore 19 % du CPU dans l'allocateur et le GC, avec 1,2 Mio et 6 440 allocations par tick. Pour la V2 j'ai remplacé la copie de `RowItems` par une sous-tranche du tableau, j'ai gardé les tampons des grilles d'un tick à l'autre et j'ai préalloué la file utilisée pour le calcul des chemins. Avec `go build -gcflags=-m` on voit que plus rien ne s'échappe sur le tas pendant le tick, par exemple le filtre passé à `Nearest` est marqué `does not escape`. Le tick baisse encore de 24 % en milieu de partie et le benchmark affiche 0 B/op, 0 allocs/op et plus aucun cycle de GC.

Pour la V3 j'ai regardé l'alignement de la struct `Agent`. Sur la version d'origine, `fieldalignment` affiche `Agent has size 88 but the optimal size is 64 leading to a waste of 32 bytes (33%)`. En triant les champs du plus grand au plus petit, un agent tient exactement dans une ligne de cache de 64 octets. Pourtant benchstat ne voit aucune différence une fois la grille en place (p = 0,80). Comme je m'attendais à un gain, j'ai refait le test sur la version O(N²), qui lit tout le tableau en continu, et là le passage à 64 octets fait gagner 12 % (de 116,4 ms à 102,3 ms avec p = 0,000). Avec la grille on ne lit que quelques agents par requête et le tableau de 562 Ko tient de toute façon dans le cache L2, donc la taille de la struct ne change presque rien. Même avec 30 000 agents l'écart reste non significatif.

Pour la V4 je suis reparti du profil de la V3, où `math.Sin` et `math.Cos` prenaient 5,8 % du temps. Au début j'avais lu 20 % parce que j'avais additionné la colonne cum de deux fonctions dont l'une appelle l'autre. `math.Sincos` calcule les deux valeurs en un seul appel. Mesurées en deux séries séparées, les deux versions ne se distinguaient pas. En alternant les mesures des deux versions plusieurs fois, benchstat trouve un gain de 2,7 % au début de la partie (p = 0,002). Le gain est petit mais il est bien là.

4.2 Concurrence et scalabilité CPU

Pour paralléliser la décision des agents sans verrou, il fallait que chaque agent n'écrive que dans sa propre case. J'ai donc donné à chaque agent son propre générateur aléatoire, calculé à partir de la graine, du tick et de l'indice de l'agent. J'ai aussi déplacé avant ou après la phase parallèle les mises à jour que les voisins lisent, comme l'immunité et l'épuisement. Un test vérifie que 1, 2, 3, 8 ou 12 workers donnent exactement la même partie.

Le worker pool est partagé par toutes les parties du serveur et il est limité à GOMAXPROCS moins 1 goroutines. Les agents sont distribués par lots de 128 grâce à un compteur `atomic.Int64` et chaque worker prend le lot suivant quand il a fini le sien. J'ai comparé avec un découpage fixe en 4 parts égales, qui est 9 % plus lent avec 4 workers et 36 % plus lent avec 8, parce que tout le monde attend le worker le plus lent.

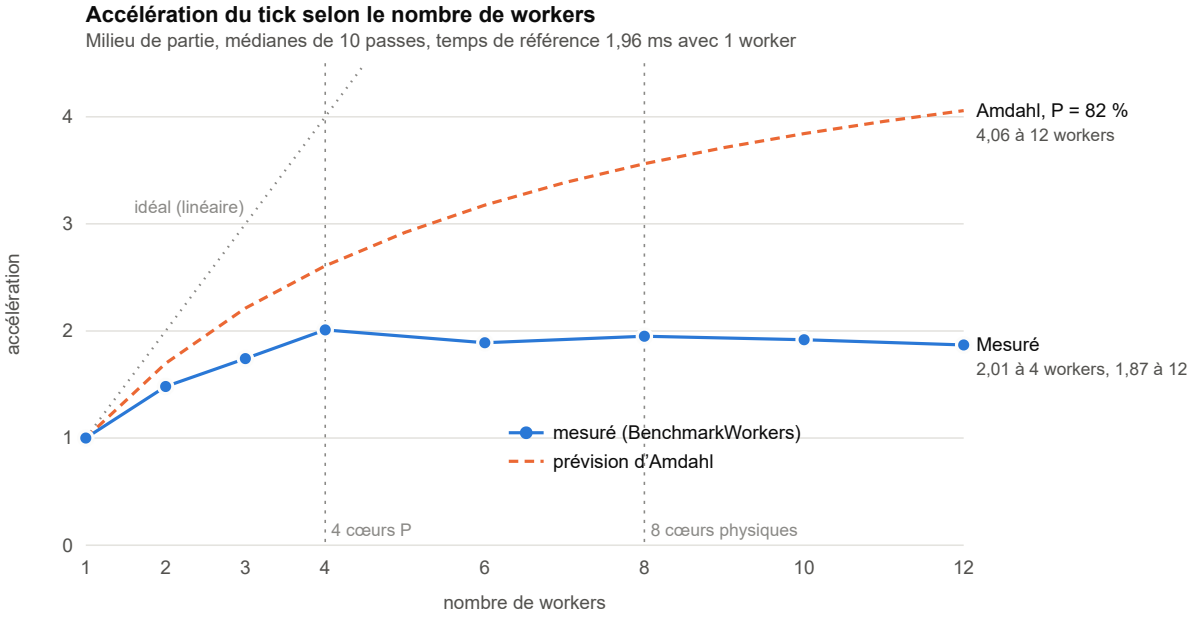


Figure 3. Accélération selon le nombre de workers comparée à la loi d'Amdahl

Le tick va 2 fois plus vite avec 4 workers, puis ça ne bouge plus. D'après le profil de la V4, 82 % du tick est parallélisable, donc la loi d'Amdahl prévoyait une accélération de 2,6 avec 4 workers. L'écart vient surtout du matériel, car il n'y a que 4 cœurs performants et au-delà on tombe sur l'hyper-threading ou sur les cœurs économes. Il faut aussi compter les 3 synchronisations par tick et les 18 % du tick qui restent séquentiels. J'ai donc dimensionné le pool sur les cœurs performants avec 4 workers.

Pour l'arrêt précoce, les parties de tournoi vérifiaient l'annulation de leur `context` seulement tous les 300 ticks. Maintenant `context.AfterFunc` met à jour un `atomic.Bool` que la partie lit à chaque tick. Arrêter un tournoi prend en moyenne 2,8 ms au lieu de 736 ms.

4.3 Réseau et base de données

En JSON, une trame avec les 9 000 agents faisait 105 Kio. Le nouveau format envoie un petit en-tête JSON suivi des positions en `int16` brutes, que le navigateur lit directement avec un `Int16Array` sans les décoder. Les tampons sont recyclés avec un `sync.Pool` et je ne convertis plus le `[]byte` en `string` avant l'envoi.

Une trame de 9 000 agents	JSON	Binaire
Temps de production	398 µs	23 µs
Taille	105,0 Kio	53,5 Kio
Mémoire allouée	227 Kio	614 octets
Débit réseau avec 8 clients	24,4 Mo/s	12,6 Mo/s

Produire une trame est donc 17 fois plus rapide et elle est deux fois plus légère. Je n'ai pas choisi Protobuf parce que les coordonnées tiennent déjà sur 2 octets en `int16`, que des varints ne prendraient pas moins de place et qu'il faudrait une librairie en plus pour décoder dans le navigateur.

Côté base de données, les tournois enregistrent l'état de chaque tick, soit 27 000 lignes par partie et 1 080 000 lignes après un tournoi de 40 parties. Avec un `INSERT` par ligne on plafonne à 281 lignes par seconde, parce que chaque ligne coûte un aller-retour et un commit. Avec `COPY` (`pgx.CopyFrom`) on monte à 209 000 lignes par seconde. Pour lire la courbe d'une partie (900 lignes), voici ce que donne `EXPLAIN (ANALYZE, BUFFERS)` sans index puis avec l'index composite `(run_id, tick)`.

```
Parallel Seq Scan on run_ticks
  Rows Removed by Filter: 366900   Buffers: shared hit=8139
Execution Time: 25.788 ms

Index Scan using run_ticks_run_tick on run_ticks
  Buffers: shared hit=12
Execution Time: 0.153 ms
```

Sans index PostgreSQL parcourt 1,1 million de lignes pour n'en garder que 900. Vue depuis Go, la requête passe de 28,3 ms à 0,91 ms. J'ai aussi mis devant la base un cache LRU limité à 256 courbes, qui répond en 27 ns quand la courbe a déjà été lue.

5. Échec constructif

Ma première idée pour paralléliser était de lancer une goroutine par agent à chaque phase et d'attendre la fin avec un `sync.WaitGroup`. Comme il y a 3 phases par tick et 9 000 agents, cela fait 27 000 goroutines par tick.

Milieu de partie	Séquentiel (V4)	Une goroutine par agent	Pool de 4 workers (V5)
Temps par tick	1,794 ms	7,717 ms	1,086 ms
Allocations par tick	0	90 003	0
Mémoire allouée par tick	0	2,95 Mio	0

Cette version est 4,3 fois plus lente que la version séquentielle. Chaque goroutine doit être créée, placée dans la file de l'ordonnanceur puis détruite, et chacune a besoin de son propre contexte de décision. Cela fait 90 000 allocations par tick et le GC revient. En divisant l'écart de temps par le nombre de goroutines, j'obtiens environ 0,22 µs de surcoût par goroutine, alors qu'un agent ne demande qu'environ 70 ns de calcul par phase. Le coût d'organisation dépasse donc le travail utile. J'ai abandonné cette version au profit du pool borné, mais je l'ai gardée dans le code pour pouvoir refaire la mesure avec `BenchmarkSpawnPerAgent`.

J'ai aussi essayé de mettre autant de workers que de threads, c'est-à-dire 12. Entre 4 et 12 workers le tick reste entre 0,98 et 1,05 ms, donc il n'y a aucun gain alors que le programme utilise trois fois plus de CPU. C'est pour ça que je suis resté à 4 workers.

6. Reproductibilité et synthèse

Toutes les mesures se relancent avec une seule commande, dans Git Bash sous Windows ou dans un terminal Linux ou macOS.

```
./run_benchmarks.sh
make bench
RAPIDE=1 ./run_benchmarks.sh
```

La première commande lance tout en 45 minutes environ, la deuxième fait la même chose avec le Makefile et la troisième est une version courte pour vérifier que tout fonctionne. Le script vérifie que les outils sont installés, note l'environnement et lance les tests. Il compile ensuite chaque version avec ses tags puis enchaîne les benchmarks, benchstat, hyperfine, les profils pprof, les flamegraphs, un test de charge WebSocket et la partie PostgreSQL avec `docker compose`.

hyperfine sur 300 ticks	Moyenne	Médiane	Écart-type	Variance	Min	Max	Gain
V0 naïve	34,394 s	34,355 s	0,523 s	0,273 s²	33,61 s	35,21 s	x1
V1 grille	0,731 s	0,719 s	0,041 s	0,00168 s²	0,68 s	0,82 s	x47
V2 zéro alloc	0,589 s	0,589 s	0,007 s	0,0000488 s²	0,58 s	0,61 s	x58
V3 64 octets	0,593 s	0,593 s	0,004 s	0,0000151 s²	0,59 s	0,60 s	x58
V4 sincos	0,589 s	0,582 s	0,022 s	0,000461 s²	0,58 s	0,65 s	x58
V5 4 workers	0,276 s	0,273 s	0,007 s	0,0000525 s²	0,27 s	0,30 s	x125

Synthèse	V0	Version finale	Gain
Tick au début de la partie	135,8 ms	0,880 ms	x154
Tick au milieu de la partie	117,5 ms	1,086 ms	x108
300 ticks avec hyperfine	34,39 s	0,276 s	x125
Mémoire allouée par tick	105,8 Mio	0	
Cycles de GC par tick	47	0	
Production d'une trame réseau	398 µs	23 µs	x17
Écriture en base	281 lignes/s	209 000 lignes/s	x745
Lecture d'une courbe	28,3 ms	0,91 ms et 27 ns en cache	x31

Au final le serveur pourrait tourner à plus de 1 100 TPS alors que le jeu n'en demande que 30. La plus grosse partie du gain vient de la grille spatiale, et le parallélisme comme la suppression des allocations apportent chacun un peu moins d'un facteur 2.

7. Bonus avec le fichier constitution.md

J'ai ajouté à la racine du dépôt un fichier `constitution.md` à donner comme prompt système aux assistants IA comme Copilot, Cursor ou Claude Code. Il suit les quatre directives vues en cours.

La première partie définit le rôle de l'IA, qui doit se comporter comme l'ingénieur système du moteur. Elle connaît les caractéristiques de ma machine, le budget de 33,3 ms par tick et les fonctions du hot path.

La deuxième partie liste les interdits. Il ne faut pas de `fmt.Sprintf` dans le hot path, pas de goroutine lancée dans une boucle sur les agents, pas de conversion entre `string` et `[]byte`, aucune allocation pendant un tick, pas de verrou dans `Update` et pas de requête SQL ligne par ligne.

La troisième partie impose une preuve. Chaque proposition doit donner une hypothèse sur l'effet matériel, la commande exacte pour la mesurer et un critère chiffré, avec p inférieur à 0,05 sur benchstat. J'y ai ajouté deux règles qui viennent de mes propres erreurs, mesurer en alternance quand le gain attendu est petit et ne pas additionner la colonne cum d'un profil.

La dernière partie fixe la forme des réponses. Les règles sont courtes et vérifiables, et l'IA doit répondre par un diff suivi de sa preuve, sans texte autour.